Building with Foundation Models

Capabilities, product choices, use cases, and the AI engineering stack

Start with the user problem and its usefulness threshold. Adapt an existing model where possible, then invest in evaluation, context, data, and operations to make the application dependable.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 1–48 / PDF p. 25. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 The Rise of AI Engineering

Source: book pp. 2–16 / PDF p. 26

From language models to large language models

A language model learns statistical relationships among tokens. Text generation repeatedly predicts the next token from prior context. Tokens need not equal words: subwords, punctuation, and bytes allow a finite vocabulary to represent unfamiliar words. Different tokenizers produce different counts, especially across languages, so token budgets and prices must be interpreted with the tokenizer in mind.

Masked models predict hidden tokens using surrounding context; **autoregressive models** predict continuations using only preceding tokens. Many tasks can be framed as completion, but raw completion is not the same as helpful conversation.

Self-supervision removes a labeling bottleneck

Targets are derived from the training data rather than written separately by human annotators. A sequence such as "We build useful tools" supplies prefix-to-next-token examples. Beginning/end tokens teach starts and stops. A loss penalizes assigning too little probability to the observed next token.

This differs from unsupervised learning with no explicit target. Large-scale pre-training becomes possible because raw sequences provide targets automatically. Human annotation is still important for later instruction and preference training, evaluation, and quality control.

From LLMs to foundation models

A foundation model is trained broadly enough to support adaptation to many downstream tasks. It can be language-only or multimodal; multimodality is not a requirement for the term. Images, audio, video, and other data can condition outputs as well as text.

CLIP illustrates natural-language supervision: paired captions and images train compatible representations. An embedding is a numerical representation designed to capture useful relationships. Joint embedding spaces let text and images be compared, supporting cross-modal retrieval and other tasks.

From foundation models to AI engineering

Broader capabilities, increased investment, and lower access barriers help explain adoption. Model APIs and available weights let teams build applications without first financing pre-training. Easier demos do not eliminate the harder work of reliability, evaluation, privacy, and product integration.

The three recurring adaptation choices are **prompting** for instructions/examples, **RAG** for retrieving relevant information into context, and **fine-tuning** for changing learned behavior. Tools and agents extend context construction into action-taking workflows.

---

02 Foundation Model Use Cases

Source: book pp. 16-28 / PDF p. 40

Coding, image/video production, and writing

- **Coding:** completion, explanation, debugging, translation, documentation, tests, and assistance across development workflows. Check execution and integration rather than judging code by appearance.
- **Image/video:** generate or edit assets, visualize ideas, create marketing material, and personalize production. Consistency, rights, brand safety, and human review remain relevant.
- **Writing:** draft, edit, brainstorm, translate, or adapt tone. Evaluate factuality and audience fit; fluent text may still be generic or wrong.

Reported productivity improvements depend on task difficulty and study conditions. Automation of individual tasks does not by itself establish replacement of an entire profession.

Education, bots, aggregation, organization, automation

- **Education:** personalized explanations, exercises, feedback, and practice. Preserve learning and verify correctness rather than merely supplying plausible answers.
- **Conversational bots:** support, assistance, entertainment, role-playing, and companionship, each with different safety and success criteria.
- **Information aggregation:** search across documents and communication channels, synthesize research, and answer source-grounded questions.
- **Data organization:** extract structured fields, classify, tag, organize media, and support analysis.
- **Workflow automation:** connect steps and tools to carry out tasks, with approvals for consequential changes.

Enterprise risk and bounded tasks

Internal applications often have a different risk profile from public-facing products. A model can be open-ended while the application task is bounded, such as routing support requests into categories. Bounded outputs are often easier to evaluate and control.

Judge the whole workflow: an impressive output is not useful if people must spend more time verifying or repairing it than the task originally required. Measure what people actually accomplish.

---

03 Planning AI Applications

Source: book pp. 28-35 / PDF p. 52

Use-case evaluation and defensibility

Clarify why AI is needed, whether simpler software or an existing product solves the problem, and whether to build or buy. Separate learning experiments from products expected to produce measurable returns.

Define how AI fits: a core product or supporting feature, static or personalized behavior, suggestions for humans or autonomous execution. Decide where a human reviews, chooses, or takes over. Defensibility can come from proprietary useful data, distribution, workflow integration, and accumulated product understanding; a temporary model weakness is a fragile business foundation.

Setting expectations

Map business outcomes into measurable quality, latency, cost, privacy, and reliability requirements. Establish a **usefulness threshold**: the minimum performance required for the application to be worth using. Different user groups may have different thresholds.

A support bot answering more messages is not necessarily helping more customers. Include satisfaction and resolution, not just output volume. Define acceptable fallback behavior and what the system must not do before choosing a model.

Milestones and maintenance

Prototype success can arrive quickly while the final reliability gains take much longer. Budget for edge cases, evaluation data, deployment, feedback, and ongoing operations rather than extrapolating linearly from a demo.

Track changing prices, providers, model behavior, user demand, and regulations. Reassess build/buy and migration decisions through cost-benefit analysis. A provider update can help one task and break another, so changes need regression tests even when advertised as improvements.

---

04 The AI Engineering Stack

Source: book pp. 35-48 / PDF p. 59

Three layers and shared responsibility

Application: evaluation, prompts, context construction through retrieval/tools, and user interfaces. **Model:** modeling/training, data curation, fine-tuning, model evaluation, and inference optimization. **Infrastructure:** serving, compute/data management, monitoring, and operational support.

The boundaries overlap. Application choices such as output length influence serving costs, and infrastructure constraints influence model choice. A team needs an end-to-end view even if specialists own individual components.

AI engineering versus ML and full-stack engineering

Traditional ML often trains task-specific predictors; foundation-model applications emphasize adaptation, large-model serving, and evaluation of open-ended behavior. Data work shifts toward tokenization, deduplication, retrieval, and demonstration quality rather than disappearing.

Full-stack engineers bring product integration and fast iteration; ML engineers bring statistics, data analysis, and model knowledge. AI engineering needs both kinds of judgment. Available APIs lower the entry barrier but do not make engineering knowledge unnecessary.

Terminology: pre-training learns broad capabilities, fine-tuning continues training from existing weights, and post-training usually means shaping a pre-trained model toward instruction-following and preferences before release.

Revision Check

1. Why is a foundation model not necessarily multimodal?
2. How does self-supervision create training targets?
3. What is the usefulness threshold of your intended product?
4. Which task needs prompting, retrieval, or changed weights, and why?

A lower barrier to building a demo increases, rather than removes, the importance of product fit, rigorous evaluation, and disciplined operations.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.